

Redesign the complete 360extract Web UI

Redesign the existing 360extract Web UI into a polished professional desktop-style application for processing 360 video into Gaussian splats.

The application is not a normal website or SaaS dashboard. Treat it like professional production software similar in spirit to a video editor, photogrammetry application, or 3D processing tool.

Do not change working backend functionality unnecessarily. First inspect the existing frontend structure, CSS, templates, JavaScript, API endpoints, and job handling. Reuse the current technology stack unless there is a very strong technical reason not to.

The redesign must improve both appearance and workflow.

Core workflow

The entire application should be organized around these five stages:

1. Capture
2. Refine
3. Reconstruct
4. Train
5. Inspect

The user should immediately understand:

Video -> Extract images -> Remove unwanted objects -> Reconstruct cameras -> Train splat -> Inspect and clean result

The current three-tab structure should be replaced with this pipeline.

The existing Capture functionality is already strong and should largely be preserved. The main problem is the surrounding application structure.

The current global footer must disappear. There must never be an "Extract" button visible while the user is working in Refine, Reconstruct, Train, or Inspect.

Every stage owns:

- its own primary action
- its own secondary actions
- its own progress
- its own status
- its own cancel action
- its own log where applicable

Overall application layout

Create a persistent application shell.

Top application bar

Approximately 48 to 56 px high.

Left:

- 360extract logo/name
- current project name
- small project status indicator

Center or remaining area:

- source video filename
- resolution
- duration
- frame count where useful

Right:

- Open Project
- Save
- Settings
- System/Doctor status

Avoid a traditional menu-heavy web application look.

Use compact icon buttons where their meaning is obvious.

Pipeline navigation

Directly below the top bar, add a prominent horizontal pipeline navigator:

Capture Refine Reconstruct Train Inspect

Each stage must show its current state visually.

Possible states:

- Pending
- Ready
- Running
- Done
- Stale
- Error
- Disabled

Use both iconography and color. Do not communicate status through color alone.

Suggested visual language:

Pending: small hollow circle

Ready: subtle neutral indicator

Running: animated spinner or pulsing progress indicator

Done: checkmark

Stale: warning icon

Error: error icon

The currently selected stage should be visually stronger than the others.

Do not make these look like ordinary browser tabs. They should feel like a processing pipeline.

Connect the stages with a subtle horizontal line so the workflow direction is visually obvious.

The pipeline should remain visible at all times.

Stages that cannot currently run should still be visible but disabled. Hovering or clicking them should explain why.

Examples:

"Extract images before reconstruction can begin."

"Run reconstruction before training."

"No trained .ply file is available yet."

Use the existing project stage fingerprints and state information rather than creating duplicate frontend logic.

Main workspace

The center of the application should be dominated by the actual content being worked on.

Do not surround the content with excessive cards.

Use a structure similar to:

- large central workspace
- optional right inspector panel
- optional bottom timeline, filmstrip, or log
- stage-specific bottom action bar

The central visual content should always receive the most space.

Use resizable panels where useful, especially for:

- Capture settings
- Refine settings
- logs
- Inspect cleanup controls

Store panel sizes locally so they remain where the user left them.

Stage-specific bottom action bar

Every stage should have its own bottom action bar.

The left side should show:

- stage status
- progress bar when running
- percentage
- concise current operation

The right side should contain:

- Cancel while running
- secondary action if needed
- primary action

Examples:

Capture: Extract Frames

Refine: Run Detection

Reconstruct: Run All

Train: Start Training

Inspect: Apply Cleanup

The primary action should always be visually obvious.

Do not place unrelated actions in this bar.

When a job runs, transform the action area into a useful live status display rather than simply disabling the button.

Example:

62% Registering image 24 of 40

[Cancel]

Progress should animate continuously when actual progress information is available.

Never show a progress bar permanently sitting at zero while a process is actively working.

Visual design

Create a modern dark desktop application interface.

The design should feel technical, precise, and calm.

Avoid:

- generic Bootstrap appearance
- oversized rounded SaaS cards
- excessive gradients
- glassmorphism
- huge headings
- unnecessary empty space
- colorful dashboard widgets
- excessive animation

Use mostly flat surfaces with subtle depth.

Suggested palette:

Main application background: very dark graphite

Workspace background: almost black or very dark neutral

Panels: slightly lighter graphite

Borders: subtle neutral gray

Primary accent: clean blue or cyan

Running: blue/cyan

Success: muted green

Warning/Stale: amber

Error: muted red

Selection: accent color with a soft background tint

The panorama, images, masks, and splat must visually dominate the application. UI chrome should stay neutral.

Typography

Use the existing system font where possible, or Inter if already available.

Suggested hierarchy:

Application title: 16 to 18 px

Stage labels: 13 to 14 px medium

Panel headings: 12 to 13 px semibold

Normal controls: 12 to 13 px

Metadata: 11 to 12 px

Logs: monospace

Keep the interface compact enough for a professional desktop tool.

Spacing

Use an 8 px spacing system.

Typical values:

4 px for tightly related elements

8 px for control spacing

12 to 16 px for panel padding

24 px only between larger logical groups

Do not waste workspace area with large margins.

Controls

Inputs should be compact.

Use:

- sliders with numeric fields
- segmented controls for small option groups
- switches for binary options
- dropdowns only where appropriate
- tooltips for technical settings
- collapsible advanced sections

Every technical option should have a clear label.

Frequently changed settings stay visible.

Rare settings go under "Advanced".

Use Lucide icons instead of manually mixing Unicode symbols or unrelated icon sets.

Stage 1: Capture

Preserve the existing panorama interaction because this is currently one of the strongest parts of the application.

The main workspace should be the equirectangular panorama.

Keep:

- camera footprints
- camera aiming
- rig configuration
- occluder painting
- static mask visualization
- frame selection

Recommended layout:

Central: large panorama canvas

Right inspector: collapsible sections

Project Rig Image Occluder Output Orientation

Bottom: video/frame timeline

The selected frame time should be clearly visible.

Place extraction controls in the Capture stage action bar.

The right panel should support collapsing individual sections so the panorama can get more room.

The camera preview should either:

- appear as a floating preview panel, or
- occupy a clearly defined preview section in the inspector

Do not permanently sacrifice large amounts of canvas area for a small preview.

Stage 2: Refine

Redesign Refine as a proper mask review workspace.

Recommended layout:

Left or right inspector: Detection configuration

Center: large current image

Bottom: filmstrip of extracted frames

Optional secondary panel: detected object list and mask information

The image viewer should support:

- zoom
- pan
- fit to screen
- 100% view
- previous frame
- next frame

Masks should be clearly visible but adjustable.

Add a mask opacity control.

Allow quick switching between:

- Original
- Mask overlay
- Mask only

The current camera and frame should always be obvious.

Example:

Camera c03 Frame 24 / 240

Detection should have real progress.

Display:

Processing camera c03 Frame 24 / 40 Overall 104 / 240

The user should be able to cancel detection.

Add a collapsible live log panel.

If another job is already running, never show only:

"Something is already running."

Instead say exactly what is running.

Example:

"Dynamic mask detection is currently running."

Provide a button that takes the user to that running stage.

Stage 3: Reconstruct

Replace the current Export concept with a real Reconstruct workspace.

The main goal is to make COLMAP feel like part of the application rather than an external shell command.

The reconstruction pipeline contains:

1. Feature Extraction
2. Rig Configuration
3. Feature Matching
4. Mapping
5. Optional Geo Alignment

Create a vertical step list on the left.

Each step shows:

- status
- duration when completed
- rerun button
- error state

The user can run individual steps.

The primary action is:

Run All

The center area should contain a live log.

Use a proper monospace log viewer with:

- auto-scroll
- pause auto-scroll
- copy
- clear visual distinction between normal output, warnings, and errors

Do not turn every log line into a colored component. Keep it visually simple.

Above the log, show reconstruction progress.

When COLMAP reports image registration progress, display:

Registering images 24 / 40 60%

After completion, replace the progress summary with reconstruction results.

Show useful metrics in a compact result strip:

Registered Images 240 / 240

Sparse Points [number]

Mean Reprojection Error 0.54 px

Rig Spread [value]

Do not use oversized dashboard cards. Use compact metric blocks.

Add an optional geo-registration section when GPX data is available.

Keep "Generate Script" as a secondary advanced action.

It should no longer be the main workflow.

Stage 4: Train

Create a dedicated training workspace for Brush.

Recommended layout:

Left inspector: Training settings

Center: training status and live log

Top of workspace: large training progress summary

Settings should include at least:

- total steps
- maximum resolution
- export interval
- export path
- evaluation split
- launch with viewer

Separate basic and advanced settings.

When training runs, show:

Current Step 12,420 / 30,000

Progress 41%

Elapsed Time

Estimated Remaining Time, only if a reasonably stable estimate can be calculated

Current output or checkpoint if available

Show the Brush log below this.

Training should be cancellable.

When training completes:

- clearly mark Train as Done
- discover the generated .ply automatically
- pass the result directly to Inspect
- make "Open in Inspect" the obvious next action

The user should not have to browse for a file that the application just created.

Stage 5: Inspect

This should feel like the final destination of the pipeline.

The Gaussian splat viewer should occupy almost the entire workspace.

Serve and embed the existing local SuperSplat build.

Do not open it as a completely separate unrelated application if it can be integrated cleanly.

Recommended layout:

Center: SuperSplat viewer

Right inspector: Splat Cleanup

Bottom or floating minimal viewer toolbar: viewer controls

The cleanup panel should contain the existing cleanup controls:

- radius
- floor
- up direction
- related path cleanup options

Create two distinct cleanup operations:

Preview Cleanup

Apply Cleanup

Preview Cleanup should produce or load the removed points separately so the user can visually inspect exactly what would be deleted.

Provide a simple viewer mode switch:

Result

Removed Points

Comparison

If technically practical, allow toggling the removed points overlay directly over the original splat.

The user should be able to verify cleanup before permanently writing the cleaned result.

Show a concise cleanup summary:

Gaussians Before 318,000

Would Remove 5,942

Remaining 312,058

Again, use compact metric blocks rather than giant cards.

Job system UI

Update the frontend architecture to understand independent jobs by stage.

Each stage job should expose:

- state
- fraction
- message
- log
- cancellable
- error

Create a shared frontend job component used by:

- Capture
- Refine
- Reconstruct
- Train
- Inspect cleanup

Do not implement five unrelated progress systems.

The visual behavior should be consistent everywhere.

When navigating away from a running stage, the process must continue.

The pipeline navigation should continue showing that stage as Running.

Returning to the stage should immediately restore:

- current percentage
- current message
- complete recent log

Responsive behavior

The application is primarily a desktop tool.

Optimize first for:

1920x1080

2560x1440

4K displays

It should remain usable around 1366x768.

Do not redesign it as a mobile application.

Use CSS Grid and Flexbox for the major layout.

Use container queries for components that need to adapt based on panel width.

At narrower desktop widths:

- inspector panels may collapse
- labels may become icons where safe
- pipeline spacing may become tighter

Do not hide core functionality.

Implementation tools

Use the current frontend technology already present in the repository.

Do not migrate the application to React, Vue, Svelte, or another framework just for this redesign.

Use:

- CSS custom properties for design tokens

- CSS Grid for the application shell and workspaces
- Flexbox for toolbars and control rows
- container queries for adaptive panels
- Lucide for interface icons
- existing backend communication mechanisms for job updates

If the current application uses plain JavaScript, keep it modular and create reusable UI modules rather than introducing a large framework.

Create reusable components or equivalent modules for:

- PipelineStage
- StageActionBar
- JobProgress
- LogViewer
- InspectorSection
- MetricStrip
- EmptyState
- DisabledStageReason
- StatusBadge

Names can be adapted to the existing architecture.

For the SuperSplat integration, use the existing built files in:

C:\Tools\supersplat\dist

Serve them through the application's own server and integrate the viewer into Inspect.

Do not add a second large 3D viewer dependency unless the existing SuperSplat build cannot be embedded.

Design tokens

Create one centralized set of CSS variables.

At minimum define:

- application background
- workspace background
- panel background
- elevated panel background
- border colors
- primary text
- secondary text
- disabled text
- accent
- success
- warning
- error
- spacing values

- panel radius
- control radius
- toolbar heights
- inspector width

Do not scatter arbitrary colors and spacing values throughout the CSS.

Keep corner radii modest.

Approximately:

4 to 6 px for controls

6 to 8 px for panels

Avoid excessive rounded rectangles.

Interaction details

Add polished small interactions where useful:

- smooth panel collapse
- subtle hover feedback
- clear pressed states
- visible keyboard focus
- temporary success confirmation after actions
- informative tooltips

Avoid decorative animation.

Running indicators can animate.

Nothing else needs to constantly move.

Use proper semantic buttons and accessible tab behavior for the pipeline navigation.

Keyboard interaction should remain usable.

Error handling

Errors from processing stages must appear inside the relevant stage.

Do not rely on temporary toast notifications for important failures.

A failed reconstruction should remain visibly marked as Error until resolved.

Display:

- concise error summary

- expandable technical details
- relevant log tail
- retry action

The pipeline navigation should also show the failed stage.

Empty states

Create intentional empty states.

Examples:

No video loaded

No extracted frames

No masks generated

No sparse reconstruction

No trained splat

Each empty state should explain the next required action and provide the relevant action when possible.

Do not leave large blank panels.

System status

The existing doctor/binary detection functionality should be available from a compact system status dialog.

Show:

FFmpeg COLMAP Brush SuperSplat

Each should show:

Detected Missing Configured manually

Allow path overrides where supported.

Do not occupy permanent main workspace area with these diagnostics.

Testing

After implementation, use Playwright to test the complete major workflow at the UI level.

At minimum cover:

- project opens
- video loads
- Capture stage becomes available
- Extract action belongs only to Capture
- running extraction shows progress
- Refine detection shows actual progress
- detection can be cancelled
- running jobs remain visible after changing stages
- Reconstruct correctly gates unavailable actions
- COLMAP steps show their state
- Train is unavailable before reconstruction
- Inspect is unavailable before a .ply exists
- completed .ply automatically appears in Inspect
- stage states survive page reload where project state supports it

Add screenshot tests for the five major stages at a standard desktop resolution.

Run accessibility checks for the main application shell and primary controls.

Important implementation rule

Do not perform a cosmetic reskin of the existing three tabs.

The architecture of the interface must reflect the actual pipeline.

The current UI problem is fundamentally structural:

Capture, Refine, reconstruction, training, and cleanup are different stages with different actions and different long-running jobs.

The new UI should make that relationship obvious at all times.

Implementation order

Work in this order:

1. Inspect the current frontend and map all existing controls and endpoints.
2. Introduce the new shared application shell and design tokens.
3. Build the five-stage pipeline navigation.
4. Introduce the reusable stage action bar and job UI.
5. Move existing Capture functionality into the new Capture workspace without breaking its canvas interaction.
6. Redesign Refine and connect real progress and cancellation.
7. Build Reconstruct around actual COLMAP execution.
8. Build Train around Brush.
9. Integrate SuperSplat into Inspect.
10. Move splat cleanup from the old Export page into Inspect.
11. Remove obsolete Export UI and the global Extract footer.

12. Add responsive behavior and final visual polish.
13. Add Playwright workflow tests.

Before deleting old UI code, verify that every existing setting and action has a new location.

Final quality target

The finished application should feel like one coherent professional 360 reconstruction tool.

A user should be able to open it with no documentation and immediately understand:

where they are, what has already been completed, what is currently running, what they need to do next, and where the final Gaussian splat will appear.

Prioritize the central visual workspace and the pipeline itself.

The redesign should make 360extract feel like a complete application rather than several processing tools collected behind browser tabs.